SLIDING WINDOW

Day 3 — The Universal Variable-Window Template

Three Classic Problems Solved with ONE Template — Exactly K Distinct, Fruits Into Baskets & No Repeating Characters

---

DSA Pattern Series — Sliding Window, Part 3

Covers: Longest Substring with Exactly K Distinct + Fruit Into Baskets + Longest Substring Without Repeating Characters

Compiled from video transcript + added explanations, tables & code

Table of Contents

1. Quick Recap & An Important Correction

Day 2 covered two problems and introduced the 4-step approach to any Sliding Window problem (Identify → Window Type → First Window's Info → Slide & Update). One small but important fix before going further:

🔧 Correction from Day 2

The high pointer should be driven by a simple for loop (for high = 0 to n-1), NOT a while loop.

A while loop made it look like high needed special handling — it doesn't. high always just moves forward by one, every single iteration, no exceptions.

Only low ever needs conditional/loop-based movement. Keeping this separation clean is what makes the final template reusable everywhere.

Why does this matter so much? Because today's goal is to nail down ONE template that solves every variable-window problem with just a line or two changed. A messy for/while mix-up makes that template harder to reuse correctly.

2. The Final, Polished 3-Step Variable-Window Template

This is the cleaned-up, final version of the variable-window approach. Every problem in this document — and most variable-window problems you'll ever see — is solved by filling in the blanks of this exact skeleton.

Step 1 — Include high in the Window's Information

high moves forward every single time, with no exceptions, via a plain for loop. The very first thing you do inside that loop is add arr[high] (or s[high]) into your information structure (a running sum, a frequency hashmap, etc.).

Step 2 — While the Information Is Invalid, Shrink from low

Check whatever condition the problem defines as "valid". If the information is currently invalid, remove arr[low] from the information, move low forward, and check again — repeat (in a while loop) until the information becomes valid again, or until shrinking stops being useful.

Step 3 — If the Information Is Valid, Update the Result

Once Step 2 finishes, check if the information satisfies the problem's requirement. If it does, calculate the window's length (high − low + 1) and update your result (max or min, depending on what's asked).

The Universal Variable-Window Skeleton

```
int low = 0;
// initialize an information structure: sum, frequency map, etc.
// initialize result appropriately (0, INT_MIN, or INT_MAX)

for (int high = 0; high < n; high++) {
    // STEP 1: always include high
    addToInfo(arr[high]);

    // STEP 2: shrink while info is invalid
    while (infoIsInvalid()) {
        removeFromInfo(arr[low]);
        low++;
    }

    // STEP 3: if info is valid, update the result
    if (infoIsValid()) {
        int length = high - low + 1;
        result = max(result, length);   // or min(...), depending on the ask
    }
}
return result;
```

Why Is This O(N) and Not O(N²)?

It looks scary — a for loop with a while loop nested inside — but it is NOT O(N²). Here's the proof:

- high moves forward exactly N times total — once per outer iteration. No surprises there.
- low ALSO only ever moves forward — it never resets and never goes backward.

- Because both pointers only move forward and **low can never cross high**, the total number of times low moves across the ENTIRE run of the algorithm — added up over every outer iteration — is still capped at N.

The Real Complexity

Total work = (N moves of high) + (at most N moves of low, spread across the whole run) = O(N) + O(N) = O(N).

This style of reasoning is called amortized analysis, and it's a very common talking point in interviews when someone challenges you about nested loops.

3. Problem 1 — Longest Substring with Exactly K Distinct Characters

🎯 Problem Statement

Given a string of lowercase characters and an integer K, find the length of the LONGEST substring that contains EXACTLY K distinct characters.
String: "aabbcc" K = 2

3.1 Flowchart Check

Check	Is it true here?
Array / String question?	✅ Yes — it's a string
Substring (contiguous)?	✅ Yes — "longest substring"
Has a K-related keyword?	✅ Yes — "exactly K distinct"

Window type: since no fixed length is given (the answer's length is exactly what we're solving for), this is a VARIABLE window.

3.2 What Is the "Information" Here?

We need to know how many DISTINCT characters are currently inside the window. The natural tool for this is a frequency hashmap: every time a character enters the window, increase its count; every time it leaves, decrease its count (and erase it entirely once its count hits zero, so the map's size always reflects only the characters truly still present).

```
Information = a hashmap of {character → frequency}
Number of distinct characters in the window = hashmap.size()
```

3.3 Deciding When to Shrink — The Three Possible States

hashmap.size() vs K	Meaning	Should we shrink (move low)?
size == K	Perfect — exactly K distinct characters	No — this is already valid; record it
size < K	Too FEW distinct characters so far	No — shrinking only ever keeps size the same or REDUCES it further, moving AWAY from K. We need high to grow instead.
size > K	Too MANY distinct characters	Yes — shrinking can reduce the size back down toward K.

The Core Insight

Moving high can only ever INCREASE the information's size (or keep it the same).
Moving low can only ever DECREASE the information's size (or keep it the same).
So: shrink only when you're currently ABOVE your target — shrinking when you're already below it can never help.

3.4 Step-by-Step Walkthrough

String: "aabbcc" with indices 0(a) 1(a) 2(b) 3(b) 4(c) 5(c). K = 2. low = 0.

high	Char	Map After Include	Map Size	Shrink?	low After	Size == K?	Length	Result
0	a	{a:1}	1	No (< K)	0	No	—	0
1	a	{a:2}	1	No (< K)	0	No	—	0
2	b	{a:2,b:1}	2	No (= K)	0	Yes	3	3
3	b	{a:2,b:2}	2	No (= K)	0	Yes	4	4
4	c	{a:2,b:2,c:1}	3	Yes (> K) → twice	2	Yes	3	4
5	c	{b:2,c:2}	2	No (= K)	2	Yes	4	4

Note row 4 ("Shrink? Yes... twice"): adding 'c' pushes size to 3 (a, b, c). The while loop fires twice — first removing index 0 ('a', frequency drops from 2 to 1, 'a' is NOT erased yet), then removing index 1 ('a' again, frequency hits 0, NOW it's erased) — bringing size back down to 2 (b, c) and low to 2.

Result

Longest substring with exactly 2 distinct characters = 4 (the window "aabb")

3.5 C++ Code

C++ — Variable Window | Time: O(N) | Space: O(min(N, 26)) ≈ O(1)

```
#include <bits/stdc++.h>
using namespace std;

int longestSubstringExactlyKDistinct(string s, int k) {
    int n = s.size();
    unordered_map<char, int> freq;
    int low = 0;
    int result = 0;

    for (int high = 0; high < n; high++) {
        // Step 1: always include high
        freq[s[high]]++;

        // Step 2: shrink ONLY when we have TOO MANY distinct characters
        while ((int)freq.size() > k) {
            freq[s[low]]--;
            if (freq[s[low]] == 0) freq.erase(s[low]);
            low++;
        }

        // Step 3: only an EXACT match counts as a valid answer
        if ((int)freq.size() == k) {
            int length = high - low + 1;
            result = max(result, length);
        }
    }
    return result;
}

int main() {
    string s = "aabbcc";
    cout << "Longest substring with exactly 2 distinct chars = "
         << longestSubstringExactlyKDistinct(s, 2) << endl;
    return 0;
}
```

Output:

```
Longest substring with exactly 2 distinct chars = 4
```

4. Problem 2 — Fruit Into Baskets (At Most K Distinct)

Problem Statement (Real Interview Question)

Fruit trees are planted in a row, each producing one type of fruit (represented as a number). You have one basket, and it can hold AT MOST K distinct types of fruit (any quantity of each).

Picking must go strictly left to right with no skipping. Return the MAXIMUM total number of fruits you can collect.

Trees: [0, 1, 2, 2] K = 2

This question was asked in a real Amazon interview. The story (farms, baskets, fruit trees) is just decoration — the skill being tested is recognizing the underlying pattern underneath the story.

4.1 Translating the Story Into Algorithm Language

Strip away the story and this is exactly: "Find the length of the longest subarray that has AT MOST 2 distinct numbers." That's it — the same shape as Problem 1, with one word changed ("exactly" → "at most").

Translated Problem

Find the LONGEST subarray which has AT MOST K distinct numbers.

4.2 What Changes From Problem 1?

Aspect	Problem 1 (Exactly K)	Problem 2 (At Most K)
Data type	String of characters	Array of integers
Hashmap key type	char	int
Shrink condition	size() > K (same)	size() > K (same)
Result update condition	Needs an if: size() == K only	No if needed: size() ≤ K is ALWAYS true after shrinking

Why No "if" Is Needed Here

"At most K" accepts BOTH size == K and size < K as valid.

After the shrink loop runs, size() is guaranteed to be ≤ K (we only ever shrink while size() > K).

So every window that survives Step 2 is automatically valid — Step 3 can update the result unconditionally.

4.3 Step-by-Step Walkthrough

Trees: [0, 1, 2, 2] with indices 0, 1, 2, 3. K = 2. low = 0.

high	Tree Type	Map After Include	Map Size	Shrink?	low After	Length	Result
0	0	{0:1}	1	No	0	1	1
1	1	{0:1,1:1}	2	No	0	2	2
2	2	{0:1,1:1,2:1}	3	Yes → once	1	2	2
3	2	{1:1,2:2}	2	No	1	3	3

At high = 2, including tree type 2 pushes the map to size 3. The while loop removes tree type 0 at index 0 (its frequency hits 0, so it's erased), bringing the map back to size 2 and low to 1.

Result

Maximum fruits collected = 3 (the window [1, 2, 2] — types 1 and 2, 3 fruits total)

4.4 C++ Code

C++ — Variable Window | Time: O(N) | Space: O(K) ≈ O(1)

```
#include <bits/stdc++.h>
using namespace std;

int totalFruits(vector<int>& trees, int k) {
    int n = trees.size();
    unordered_map<int, int> freq;
    int low = 0;
    int result = 0;

    for (int high = 0; high < n; high++) {
        // Step 1: always include high
        freq[trees[high]]++;

        // Step 2: shrink ONLY when we exceed K distinct fruit types
        while ((int)freq.size() > k) {
            freq[trees[low]]--;
            if (freq[trees[low]] == 0) freq.erase(trees[low]);
            low++;
        }

        // Step 3: "AT MOST K" means every surviving window is already valid
        int length = high - low + 1;
        result = max(result, length);
    }
    return result;
}

int main() {
    vector<int> trees = {0, 1, 2, 2};
    cout << "Maximum fruits collected = " << totalFruits(trees, 2) << endl;
    return 0;
}
```

Output:

```
Maximum fruits collected = 3
```

5. Problem 3 — Longest Substring Without Repeating Characters

Problem Statement

Given a string, find the length of the LONGEST substring in which every character is unique (no duplicates at all).
String: "abcabcbb"

5.1 The Twist — There Is No K This Time

Problems 1 and 2 both revolved around comparing hashmap.size() to a given K. This problem gives you no K at all — so what do we compare against?

The Key Realization

A substring has NO repeating characters exactly when:
hashmap.size() (number of distinct characters) = window size (high − low + 1)
If a character repeats anywhere inside the window, the hashmap size will be SMALLER than the window size — because the hashmap only counts each character once, no matter how many times it repeats.
So K isn't given — it IS the current window size, recalculated every time the window changes.

5.2 Proof: hashmap.size() Can Never Exceed the Window Size

This collapses the usual 3-state check (size <, ==, > K) down to just 2 states — and the reasoning is simple:

- The window size counts EVERY character inside the window, including repeats.
- The hashmap size counts only the DISTINCT characters — at most one entry per unique character.
- So hashmap size can be EQUAL to window size (everything is unique) or LESS than window size (something repeats) — it can never be GREATER. There's nowhere for an extra distinct character to come from.

hashmap.size() vs Window Size	Meaning	Action
size == window size	Every character in the window is unique	Valid — record the length
size < window size	At least one character repeats somewhere inside	Invalid — shrink from low until it's fixed
size > window size	Impossible — can never happen	(No case needed for this)

5.3 Step-by-Step Walkthrough

String: "abcabcbb" with indices 0(a) 1(b) 2(c) 3(a) 4(b) 5(c) 6(b) 7(b). low = 0.

high	Char	Window Size (k)	Map Size	Shrink?	low After	Length	Result
0	a	1	1	No (equal)	0	1	1
1	b	2	2	No (equal)	0	2	2
2	c	3	3	No (equal)	0	3	3
3	a	4	3	Yes → once	1	3	3
4	b	4	3	Yes → once	2	3	3
5	c	4	3	Yes → once	3	3	3
6	b	4	3	Yes → twice	5	2	3
7	b	3	1	Yes → twice	7	1	3

Watch high = 6 closely: the window size becomes 4 (indices 3-6 = "abcb") but the map size is only 3 (a, b, c — 'b' repeats). The while loop removes index 3 ('a', erased — not repeated anymore), recomputes window size = 3, but map size (b, c) is still only 2 < 3, so it shrinks AGAIN — removing index 4 ('b', frequency drops but not erased) — finally reaching equality (map size 2 = window size 2) at low = 5.

Result

Longest substring without repeating characters = 3 (the window "abc")

5.4 C++ Code

C++ — Variable Window | Time: O(N) | Space: O(min(N, 26)) ≈ O(1)

```
#include <bits/stdc++.h>
using namespace std;

int longestSubstringWithoutRepeating(string s) {
    int n = s.size();
    unordered_map<char, int> freq;
    int low = 0;
    int result = 0;

    for (int high = 0; high < n; high++) {
        // Step 1: always include high
        freq[s[high]]++;

        // K isn't given -- K IS the current window size
        int windowSize = high - low + 1;

        // Step 2: shrink while a duplicate exists
        // (distinct count < window size  <=>  some character repeats)
        while ((int)freq.size() < windowSize) {
            freq[s[low]]--;
            if (freq[s[low]] == 0) freq.erase(s[low]);
            low++;
            windowSize = high - low + 1;   // recompute after every shrink
        }

        // Step 3: every window that survives Step 2 is automatically valid
        result = max(result, windowSize);
    }
    return result;
}

int main() {
    string s = "abcabcbb";
    cout << "Longest substring without repeating characters = "
         << longestSubstringWithoutRepeating(s) << endl;
    return 0;
}
```

Output:

```
Longest substring without repeating characters = 3
```

6. Side-by-Side: One Template, Three Problems

This is the entire point of today's session — see how little actually changes between all three solutions:

Aspect	Exactly K Distinct	At Most K Distinct (Fruits)	No Repeating Characters
Input type	string (char keys)	vector<int> (int keys)	string (char keys)
Target to compare against	Given K	Given K	Current window size (high−low+1)
Shrink condition	map.size() > K	map.size() > K	map.size() < windowSize
Update condition	if map.size() == K	always (no if needed)	always (no if needed)
Possible map-size states	3 (<, ==, >)	3 (<, ==, >)	2 only (== or <, never >)

The Big Takeaway

Steps 1 and 3's STRUCTURE never change. Only the exact comparison inside Step 2's while condition, and whether Step 3 needs an if, ever change.

Once this template is in your fingers, a brand-new variable-window problem becomes a 2-minute exercise of figuring out just ONE thing: what does "valid" mean for THIS problem?

7. Common Mistakes & Pro Tips

Mistakes to Avoid

- **Shrinking whenever the information is "wrong", without checking direction:** always ask first whether you're ABOVE or BELOW your target. Shrinking only ever helps when you're above it.
- **Forgetting to erase a key once its frequency hits 0:** if you don't erase it, your hashmap's size will be wrong — it'll keep counting "ghost" characters that aren't really in the window anymore.
- **Using a while loop to move high:** high should always be a plain for loop. Only low ever needs a while loop, because only low's movement is conditional.
- **Forgetting to recompute the window size after each shrink (Problem 3 specifically):** since K is the window size itself here, it changes every time low moves — recalculate it inside the while loop, not just once before it.
- **Adding an unnecessary if-check for "at most K" problems:** after the shrink loop, the window is ALREADY guaranteed valid — wrapping the result update in an if (size == K) would silently throw away valid smaller-than-K windows.

Pro Tips

- When a problem describes a story (farms, baskets, warehouses, etc.), translate it into bare algorithmic language FIRST — strip the story, find the underlying "longest/shortest subarray/substring with property X" shape.
- Before writing code, always ask: what are the 2 or 3 possible relationships between my information and my target? Decide explicitly what to do in each one.
- "Exactly K" almost always needs an if-check at the end; "At most K" almost never does, because every window surviving the shrink step is already valid.
- If a problem doesn't give you a number to compare against, look for what naturally plays that role — often it's the window's own size, as in Problem 3.

8. Complexity Cheat-Sheet

Problem	Time Complexity	Space Complexity
Longest Substring with Exactly K Distinct	O(N) — amortized	O(min(N, 26)) ≈ O(1)
Fruit Into Baskets (At Most K Distinct)	O(N) — amortized	O(K) ≈ O(1)
Longest Substring Without Repeating Characters	O(N) — amortized	O(min(N, 26)) ≈ O(1)

All three look like they might be $O(N^2)$ at first glance because of the nested while loop — but as proven in Section 2, low's total movement across the ENTIRE run never exceeds N, so the real complexity is O(N).

9. Practice Problems

1. Longest substring with at most K distinct characters (the string version of Problem 2)
2. Subarrays with exactly K different integers (a harder twist on "exactly K" — try: exactly(K) = atMost(K) − atMost(K−1))
3. Longest repeating character replacement (allow up to K character changes)
4. Minimum window substring (variable window + two frequency maps)
5. Count number of substrings with exactly K distinct characters

10. Interview Q&A

Q: What are the 3 steps of the final variable-window template?

A: 1) Always include the element at high into the information structure (every iteration, via a plain for loop). 2) While the information is invalid, shrink by removing the element at low and moving low forward, repeating until valid (or until shrinking would no longer help). 3) If the information is valid, compute the window length (high − low + 1) and update the result.

Q: Why does moving high only ever increase a frequency-map's size, while moving low only ever decreases it?

A: Moving high always ADDS a new element into the window, so the information set can only grow or stay the same. Moving low always REMOVES an element from the window, so the information set can only shrink or stay the same. This asymmetry is exactly why shrinking is only useful when you're currently ABOVE your target, never when you're below it.

Q: Why doesn't "Fruit Into Baskets" need an if-check before updating the result, while "Exactly K Distinct" does?

A: "At most K" accepts any size from 0 up to K as valid, and the shrink loop guarantees size() ≤ K is always true once it exits — so every surviving window already qualifies. "Exactly K" only accepts one specific value, so after shrinking the size could still be less than K, which would be an invalid window to record — hence the explicit if (size == K) check.

Q: In Longest Substring Without Repeating Characters, why can the hashmap's size never exceed the window's size?

A: The window size counts every character position, including repeats. The hashmap size counts only distinct characters — at most one entry no matter how many times a character repeats. Since every distinct character is also one of the window's positions, the count of distinct characters can never be more than the total count of positions.

Q: Why is the overall complexity O(N) and not O(N²), given the nested while loop inside a for loop?

A: Although the while loop is nested inside the for loop, low never resets and never moves backward — it only ever moves forward, exactly like high does. So while low might move 0 times on some outer iterations and several times on others, its TOTAL movement summed across the entire run of the algorithm is still bounded by N. The combined work is O(N) + O(N) = O(N), known as amortized analysis.

Q: How would you adapt this template for a real-world story problem like Fruit Into Baskets?

A: First strip away the story entirely and restate the problem in pure algorithmic terms — here, "longest subarray with at most K distinct values". Once restated this way, it becomes immediately recognizable as the same shape as a problem you've already solved, and the same template applies almost unchanged (just swapping the data type from characters to integers).

Q: What's the general rule for deciding whether the result update needs an if-check at all?

A: If "valid" means the information EXACTLY equals some target (like exactly K distinct), you need an if-check, because the shrink loop alone can leave you below the target. If "valid" means the information is AT MOST or AT LEAST some target reached via shrinking in the matching direction, the shrink loop's exit condition already guarantees validity, so no extra if-check is needed.

11. One-Page Quick Revision

Sliding Window — Day 3 Cheat Sheet

1. high: always a plain for loop, 0 to n-1, no exceptions. low: only moves inside a while loop, conditionally.
2. The 3 Steps: Include high → While invalid, shrink low → If valid, update result.
3. Shrink rule: only shrink when you're ABOVE your target. Below target? Let high grow instead.
4. "Exactly K" needs an if-check on the result update. "At most / At least K" usually doesn't.
5. No K given? Look for what naturally plays that role — often it's the window's own size (high − low + 1).
6. Always erase a hashmap key once its frequency hits 0 — otherwise size() lies to you.
7. Despite the nested loop, it's all O(N) — low's total movement across the whole run is capped at N (amortized analysis).

With this template locked in, almost any "longest/shortest substring or subarray with property X" question becomes a quick exercise: figure out what "valid" means, plug it into Steps 2 and 3, and you're done.